Rafael Barbosa

_Desenvolvimento de Software e Operações de TI – DevOps

UNIDADE III



Sumário

1. Infraestrutura como código 3

2. Introdução ao AWS CloudFormation 8

3. Modificação de modelos AWS CloudFormation 28

4. Conceitos de automação 40

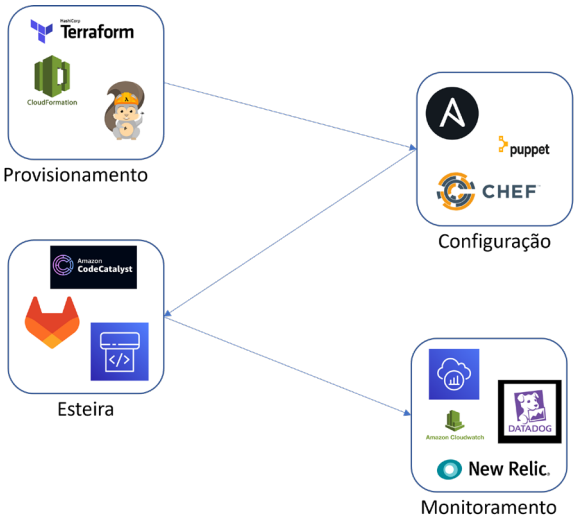
1

Infraestrutura como código

Conceitos de automação

Estágios

Figura – Estágios



Fonte: elaborada pelo autor

Provisionamento – Infraestrutura como código

Problema

Você precisa de um *software* implantado na nuvem e já decidiu usar um provedor de infraestrutura como serviço (IaaS – um dos muitos que são acrônimos), como AWS ou o Azure. Se precisar de um grande conjunto de infraestrutura, para, por exemplo, um aplicativo distribuído complexo, por exemplo, logo você se verá gastando seu tempo livre e seus fins de semana no console da AWS.

Infraestrutura como código, em termos simples, é um meio pelo qual podemos escrever definições declarativas para a infraestrutura que desejamos provisionar e usá-las como uma ferramenta de provisionamento que executa a implantação. Isso significa que podemos codificar o que queremos, fornecer as credenciais necessárias para o provedor de IaaS, iniciar o processo de provisionamento, esperar e voltar para encontrar todos os seus serviços em execução na nuvem... ou uma tela de terminal cheia de avisos ameaçadores sobre falha na implantação e "estado irrecuperável" e uma

crescente sensação de desconforto (mas isso não frequentemente, não se preocupe).

_Vantagens

- **Velocidade:** fala por si só: a automação é melhor do que navegar manualmente por uma interface para implantar e conectar recursos, tudo de uma vez.



- **Confiabilidade:** com um grande conjunto de infraestrutura, torna-se muito fácil configurar incorretamente um recurso ou provisionar os serviços na ordem errada. Com o IaC, os recursos serão configurados exatamente como declarados, e as dependências implícitas/explícitas podem ser usadas para garantir a ordem de criação.



- **Experimentação:** com a facilidade com que a infraestrutura pode ser implantada, as mudanças experimentais podem ser prontamente testadas rpadiamente com recursos reduzidos para minimizar o custo. Depois de aprovado, tudo pode ser ampliado para implantações de produção.



- **Melhores práticas**: como desenvolvedores, procuramos sempre empregar as melhores práticas conhecidas de engenharia de *software* sempre que possível. Escrever código para projetar e implantar infraestrutura facilita isso na área de provisionamento em nuvem, usando técnicas estabelecidas, como escrever código modular e configurável comprometido com o controle de versão. Isso nos leva a ver a nossa infraestrutura como uma aplicação de *software* em si, e

nos desloca na direção de uma cultura de DevOps.

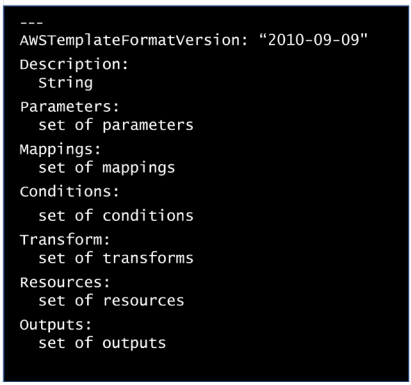


2

Introdução ao AWS CloudFormation

Estrutura do modelo AWS CloudFormation: formato YAML

YAML



```
---
AWSTemplateFormatVersion: "2010-09-09"
Description:
  String
Parameters:
  set of parameters
Mappings:
  set of mappings
Conditions:
  set of conditions
Transform:
  set of transforms
Resources:
  set of resources
Outputs:
  set of outputs
```

- **Versão do formato (opt):** versão do modelo AWS CloudFormation correspondente
- **Descrição (opt):** uma *string* de texto

- **Parâmetros (opt):** entradas no modelo
- **Mapeamentos (opt):** variáveis estáticas; pares de valores-chave
- **Condições (opt):** controles para determinar se e quando recursos específicos são criados ou atualizados
- **Transformar (opt):** especifica uma ou mais macros para processar seu modelo
- **Recursos (req):** ativos da AWS para criar
- **Saídas (opt):** valores de recursos personalizados criados por modelo (URLs, nome de usuário etc.)

Os modelos do AWS CloudFormation incluem várias seções principais, mas apenas a seção Recursos é obrigatória. Algumas seções de um modelo podem estar em qualquer ordem. No entanto, à medida que você cria seu modelo, pode ser útil usar a ordem lógica, porque os valores em uma seção podem referir-se a valores de uma seção anterior.

Para obter uma breve visão geral de cada seção do modelo, consulte (disponível em: https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/template-anatomy.html).

Parâmetros

```
InstanceTypeParameter:
    Type: String
    Default: t2.small
    AllowedValues:
      - t2.small
      - a1.xlarge
      - t3a.xlarge
      - c5n.xlarge
    Description: Enter t2.small, a1.xlarge, t3a.xlarge, or c5n.xlarge. Default
    is t2.small.
```

Use a seção opcional Parâmetros para personalizar seus modelos. Os parâmetros permitem inserir valores personalizados em seu modelo sempre que você cria ou atualiza uma pilha. Uma *pilha* é uma coleção de recursos da AWS que você pode gerenciar como uma única unidade.

Existem também tipos de parâmetros específicos da AWS que são úteis para capturar valores inválidos ao criar ou atualizar uma pilha. Para especificar parâmetros com tipos específicos da AWS, um usuário do modelo deve inserir valores existentes da AWS que estão em sua conta da AWS. O AWS CloudFormation valida esses valores de entrada em relação aos valores existentes na conta. Por exemplo, com o tipo de parâmetro AWS::EC2::VPC::Id, um usuário deve inserir um ID de nuvem privada virtual (VPC) existente que

esteja na conta e na região da AWS em que ele está criando a pilha.

_Validação de parâmetros

AllowedPattern: especifique uma expressão regular que restrinja o valor de entrada

- **Padrão permitido:** '(\\d{1,3})\\.(\\d{1,3})\\.(\\d{1,3}\\.(\\d{1,3}) (/(\\d(1,2})'

AllowedValues: especifique um conjunto de opções, como tipos de instância EC2

- **Valores permitidos:** ['t2.micro', 't2.small', 'i3.2xlarge']

Tipo de parâmetro: especifique o tipo de entrada (*string*, int) e especifique tipos específicos da AWS que validarão os elementos existentes

- AWS::EC2:: KeyPair :: KeyName

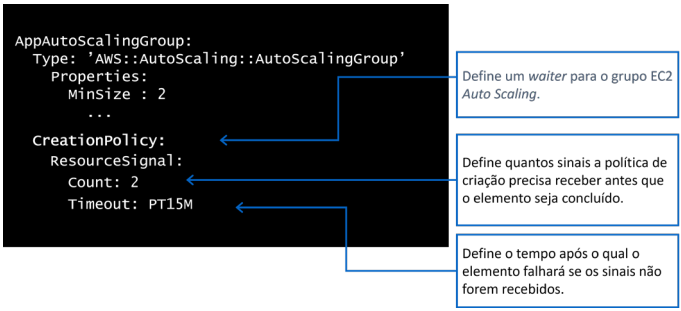
Padrão especificará um valor padrão **

Utilizar o tipo de parâmetro e a restrição corretos pode ajudar a evitar erros ao provisionar automaticamente pilhas do AWS CloudFormation.

Quando usada corretamente, a validação de parâmetros evitará a criação de desperdício de recursos (e dinheiro) quando a entrada estiver incorreta. Se o tipo de parâmetro EC2::KeyPair::KeyName for usado e um par de chaves inválido for inserido (talvez o nome de um par de chaves de outra região), a criação da pilha falhará antes de criar quaisquer recursos.

Atributo CreationPolicy

Figura – Atributo CreationPolicy



Fonte: elaborado pelo autor.

O atributo CreationPolicy é útil quando você deseja aguardar as ações de configuração de recursos antes de prosseguir com a criação da pilha. Por exemplo, se você instalar e configurar aplicativos de *software* em uma instância do Amazon EC2, talvez queira que esses

aplicativos sejam executados antes que o AWS CloudFormation crie recursos adicionais que dependam desses aplicativos. Nesses casos, você pode adicionar um atributo CreationPolicy à instância, que instruirá o AWS CloudFormation a enviar um sinal de sucesso à instância depois que os aplicativos forem instalados e configurados.

Você associa o atributo CreationPolicy a um recurso para evitar que seu *status* alcance **a criação concluída** até que o AWS CloudFormation receba um número especificado de sinais de sucesso ou o período de tempo limite seja excedido. Para sinalizar um recurso, você pode usar o *script* auxiliar cfn-signal ou a API SignalResource. O AWS CloudFormation publica sinais válidos nos eventos da pilha para que você rastreie o número de sinais enviados.

A política de criação é invocada somente quando o AWS CloudFormation cria o recurso associado. Atualmente, os únicos recursos do AWS CloudFormation que oferecem suporte a políticas de criação são AWS::AutoScaling::AutoScalingGroup, AWS::EC2::Instance e AWS::CloudFormation::WaitCondition.

Neste exemplo, você pode ver como uma política de criação é aplicada a um grupo do EC2 *Auto Scaling*. Se você especificar que deseja duas instâncias por padrão em seu grupo do EC2 *Auto Scaling*, poderá especificar que o grupo do *Auto Scaling* só será marcado como *CREATE_COMPLETE* quando receber um sinal de ambas as instâncias.

Práticas recomendadas do AWS CloudFormation

Deve girar em torno de:

- planejar e organizando suas pilhas;
- criar e validando modelo;
- gerenciar suas pilhas.

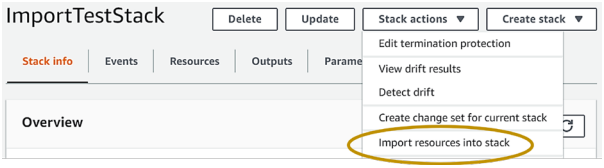
As práticas recomendadas são orientações que ajudam a usar o AWS CloudFormation de maneira mais eficaz e segura em todo o fluxo de trabalho. Aprenda como planejar e organizar suas pilhas, criar modelos que descrevem seus recursos e os aplicativos de *software* executados neles e gerenciar suas pilhas e seus recursos. Existem muitas práticas recomendadas, mas as práticas mais significativas são examinadas.

Para mais informações sobre as melhores práticas, consulte:

- Práticas recomendadas do AWS CloudFormation (disponível em: https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/best-practices.html).
- Práticas recomendadas de segurança para AWS CloudFormation (disponível em: https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/security-best-practices.html).

Pilhas AWS CloudFormation

- Crie suas pilhas por meio do console ou AWS CLI
- Os parâmetros comuns incluem:
  - ClienteRequestToken
  - OnFailure
  - Tempo limite em minutos
- Importe
  - recursos existentes



Criar uma pilha conforme especificado no modelo. Depois que a chamada for concluída com êxito, a criação da pilha será iniciada. Você pode verificar o *status* da pilha por meio da API DescribeStacks.

- **ClientRequestToken** Um identificador exclusivo para esta solicitação CreateStack. Especifique esse token se você planeja repetir solicitações para que o AWS CloudFormation saiba que você não está tentando criar uma pilha com o mesmo nome. Você pode tentar novamente as solicitações CreateStack para garantir que o AWS CloudFormation as tenha recebido com êxito.
- **OnFailure** Determina qual ação será tomada se a criação da pilha falhar. Deve ser um dos seguintes: DO_NOTHING, ROLLBACK ou DELETE. Você pode especificar OnFailure ou DisableRollback, mas não ambos.

- **TimeoutInMinutes** A quantidade de tempo que pode passar antes que o status da pilha se torne CREATE_FAILED; se DisableRollback não estiver definido ou estiver definido como falso, a pilha será revertida.

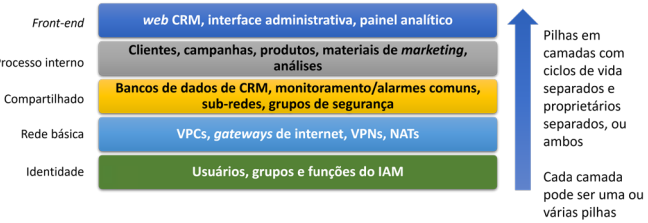
Mais exemplos de parâmetros (disponível em: https://docs.aws.amazon.com/AWSCloudFormation/latest/APIReference/API_CreateStack.html).

Durante esta operação de importação, você precisa fornecer o seguinte:

- Um modelo que descreve toda a pilha, incluindo os recursos que já fazem parte da pilha e os recursos a serem importados. Cada recurso a ser importado deve ter um atributo DeletionPolicy no seu modelo.
- Um identificador exclusivo para cada recurso de destino. Visite o console de serviço apropriado para obter identificadores exclusivos.

Pilhas: abordagem de arquitetura em camadas

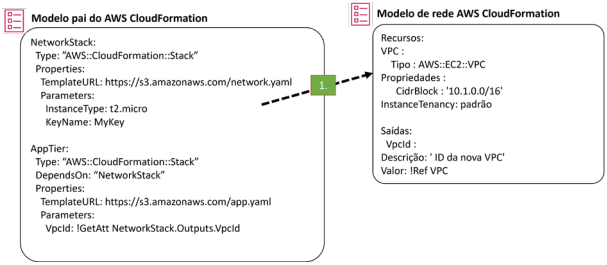
Figura – Pilhas: abordagem de arquitetura em camadas



Fonte: elaborada pelo autor.

Pilhas: camadas com pilhas aninhadas

Figura – Pilhas: camadas com pilhas aninhadas



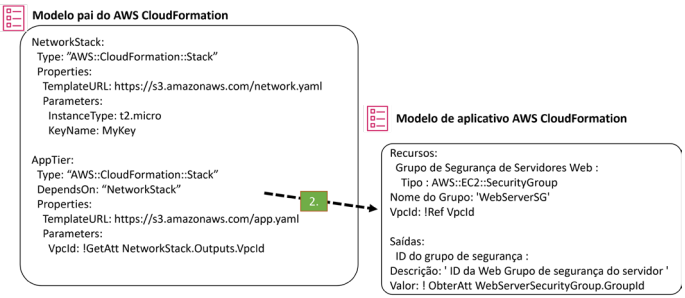
Fonte: elaborado pelo autor.

Pilhas aninhadas permitem compor modelos CloudFormation. Isso permite decompor modelos grandes em componentes reutilizáveis menores. Também ajuda a evitar limites de recursos de um único modelo. Os componentes da pilha aninhada são definidos em um modelo como qualquer outro recurso do CloudFormation.

Você pode separar esses componentes comuns e criar modelos dedicados para eles, semelhantes aos modelos de rede e aplicativo no exemplo. Dessa forma, você pode misturar e combinar modelos diferentes, mas usar pilhas aninhadas para criar uma pilha única e unificada.

1. Neste exemplo, o modelo Rede está aninhado para compor o modelo Pai.

Figura – Pilhas: camadas com pilhas aninhadas

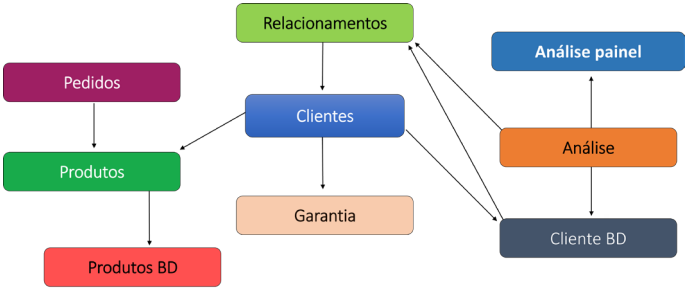


Fonte: elaborado pelo autor.

2. Neste exemplo, o modelo App está aninhado para compor o modelo pai. Observe que, no modelo pai, ! GetAtt é usado para consultar a saída VpcId do modelo de rede e passá-la como parâmetro para o modelo de aplicativo. No modelo App, !Ref é usado para referenciar o valor do parâmetro.

Pilhas: integrando pilhas independentes

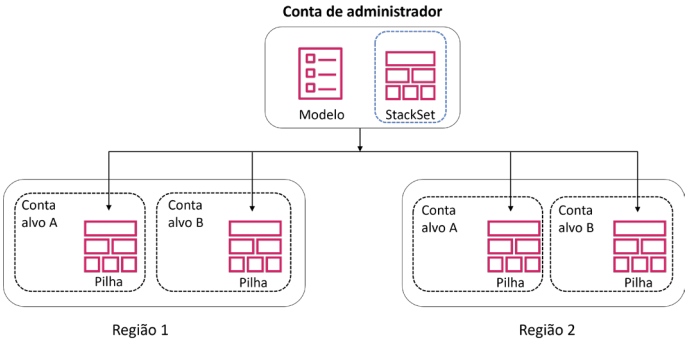
Figura – Pilhas: integrando pilhas independentes



Fonte: elaborada pelo autor.

StackSets do AWS CloudFormation

Figura – StackSets do AWS CloudFormation



Fonte: elaborado pelo autor.

O AWS CloudFormation StackSets amplia a funcionalidade das pilhas, permitindo criar, atualizar ou excluir pilhas em várias contas e regiões com uma única operação. Use uma conta de administrador para definir e gerenciar um modelo do AWS CloudFormation e use o modelo como base para provisionar pilhas em contas de destino selecionadas em regiões específicas. As operações de pilha usam uma função do IAM que você precisa criar em outras contas da AWS.

- **Conta de administrador:** conta da AWS na qual você cria conjuntos de pilhas. Um conjunto de pilhas é gerenciado fazendo *login* na conta de administrador da AWS na qual foi criado.
- **Conta de destino:** a conta na qual você cria, atualiza ou exclui uma ou mais pilhas do seu conjunto de pilhas. Antes de usar um conjunto de pilhas para criar pilhas em uma conta de destino, você deve configurar uma relação de confiança entre o administrador e as contas de destino.
- **StackSet:** com um conjunto de pilhas, você pode criar pilhas em contas da AWS em regiões usando um único modelo do

AWS CloudFormation. Ao criar o conjunto de pilhas, você especifica o modelo a ser usado, além de quaisquer parâmetros e recursos exigidos pelo modelo.

Um conjunto de pilhas é um recurso regional. Se você criar um conjunto de pilhas em uma região, não poderá vê-lo ou alterá-lo em outras regiões.

Mais informações (disponíveis em: http://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/what-is-cfnstacksets.html).

Recursos personalizados

- Inclua recursos que sejam:
  - não compatíveis como recursos padrão do AWS CloudFormation;
  - externos à AWS;
- Gerencie todos os recursos relacionados em uma única pilha.
- Especifique usando AWS::CloudFormation::CustomResource.
- Use um provedor de recursos terceirizado ou crie seu próprio recurso personalizado.

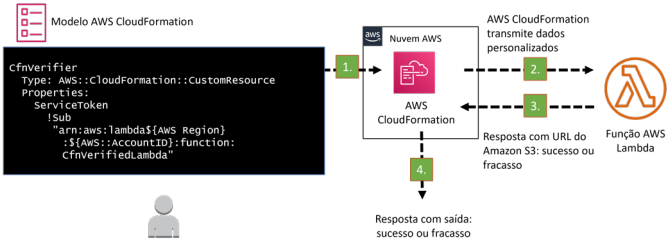
Os recursos personalizados permitem que você escreva lógica de provisionamento personalizada em modelos que o AWS CloudFormation executa sempre que você cria, atualiza (caso o recurso personalizado tenha sido alterado) ou exclui pilhas. Eles também permitem incluir recursos que não são compatíveis com os tipos de recursos padrão do AWS em uma pilha do AWS CloudFormation. Eles também permitem incluir recursos de pilha que não são compatíveis com os dois tipos de recursos padrão do AWS CloudFormation. Usando recursos personalizados, você pode adicionar todos os seus recursos que compartilham o mesmo ciclo de vida da sua pilha e gerenciá-los em uma única pilha.

Para especificar seus recursos personalizados, use AWS::CloudFormation::CustomResource ou Custom::String.

O provedor de recursos personalizados pode ser um provedor terceirizado ou você mesmo criar seus próprios recursos personalizados.

Exemplo: fluxo de trabalho de recurso personalizado

Figura – Exemplo: fluxo de trabalho de recurso personalizado



Fonte: elaborado pelo autor.

Um usuário cria um modelo do AWS CloudFormation usando uma pilha que tem uma operação de recurso personalizada definida usando AWS::CloudFormation::CustomResource ou Custom::CustomResource. O modelo inclui um *token* de serviço fornecido pelo provedor de recursos de terceiros para autenticação (*ServiceToken*) e quaisquer parâmetros definidos pelo provedor necessários para o recurso personalizado.

O AWS CloudFormation comunica-se com o provedor de recursos personalizados usando uma mensagem do *Amazon Simple Notification Service* (Amazon SNS) que inclui uma solicitação de criação, atualização ou exclusão. A mensagem

também inclui quaisquer dados de entrada armazenados no modelo de pilha e um URL do Amazon S3 para a resposta.

O provedor de recursos personalizados processa a mensagem e retorna uma resposta de sucesso ou falha ao AWS CloudFormation. O provedor de recursos customizados também poderá retornar os nomes e os valores dos atributos de recurso se a solicitação for bem-sucedida (dados de saída) ou enviar uma sequência que forneça detalhes quando a solicitação falhar.

O AWS CloudFormation define o *status* da pilha de acordo com a resposta recebida e fornece os valores de quaisquer dados de saída de recursos personalizados.

Você pode usar uma função AWS Lambda para atuar como um recurso personalizado. Para implementar isso, você pode substituir o *ServiceToken* do seu recurso personalizado pelo *Amazon Resource Name* (ARN) do seu recurso personalizado do AWS Lambda. (Em outras palavras, você não precisa criar um tópico do Amazon SNS para um recurso personalizado ao usar o AWS Lambda; o AWS CloudFormation é compatível com o AWS Lambda.)

Testando modelos do AWS CloudFormation

AWS TaskCat

- Testa modelos do AWS CloudFormation
- Desenvolvido pela equipe AWS QuickStart



cfn-lint

- Validação de recursos e melhores práticas

modelo de validação do CloudFormation

- Verifica erros de sintaxe



O TaskCat é uma ferramenta de código aberto que testa modelos do AWS CloudFormation. Ele cria pilhas em várias regiões da AWS

simultaneamente e gera um relatório com uma nota de aprovação/reprovação para cada região. Você pode especificar as regiões, indicar o número de zonas de disponibilidade que deseja incluir no teste e fornecer os valores dos parâmetros do AWS CloudFormation que deseja testar. Você pode usar um *pipeline* de CI/CD para testar qualquer modelo do AWS CloudFormation, incluindo modelos aninhados, de um repositório GitHub.

O TaskCat foi desenvolvido pela equipe AWS QuickStart. A AWS disponibiliza a ferramenta para todos os desenvolvedores que desejam validar seus modelos personalizados do AWS CloudFormation nas regiões da AWS.

Mais informações sobre o CloudFormation Linter (disponíveis em: https://github.com/aws-cloudformation/cfn-lint).

Mais informações sobre como validar um modelo – AWS CloudFormation (disponíveis em: https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/using-cfn-validate-template.html).

3

Modificação de modelos AWS CloudFormation

Atualizando pilhas

Figura – Atualizando pilhas

Atualizações sem interrupção	• O recurso é atualizado sem interromper seu funcionamento e sem alterar seu nome físico
Atualizações com alguma interrupção	• O recurso é atualizado com alguma interrupção, mas sem alteração de nome físico
Substituição	• O recurso é substituído por um recurso recém-criado. O recurso antigo será limpo (ou excluído) depois que a pilha for atualizada com sucesso

Fonte: elaborado pelo autor

O AWS CloudFormation usa uma das seguintes técnicas para atualizar recursos:

- **atualização sem interrupção:** o AWS CloudFormation atualiza o recurso sem interromper a operação desse

recurso e sem alterar o nome físico do recurso. Por exemplo, se você atualizar quaisquer propriedades em um recurso AWS::CloudWatch::Alarm, o AWS CloudFormation atualizará a configuração do alarme e, durante a atualização, a operação do alarme continuará sem interrupções;

- **atualização com alguma interrupção:** o AWS CloudFormation atualiza o recurso com alguma interrupção, mas o nome físico é mantido. Por exemplo, se você atualizar determinadas propriedades em um recurso AWS::EC2::Instance, a instância poderá sofrer alguma interrupção enquanto o AWS CloudFormation e o Amazon EC2 reconfiguram a instância;
- **substituição:** o AWS CloudFormation recria o recurso durante uma atualização, o que também gera um novo ID físico. O AWS CloudFormation cria primeiro o recurso de substituição, altera as referências de outros recursos dependentes para apontar para o recurso de substituição e, em seguida, exclui o recurso antigo. Por exemplo, se você atualizar a propriedade Engine de

um AWS::RDS::DBInstance recurso, o AWS CloudFormation cria um novo recurso e substitui o recurso atual da instância de banco de dados pelo novo.

Obtenha uma lista dos efeitos da atualização de um recurso (disponível em: http://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/aws-template-resource-type-ref.html).

Conjuntos de alterações

Use para:

- visualizar alterações propostas em uma pilha;
- determine como eles afetam os recursos em execução;
- verifique se suas alterações excluem ou substituem recursos críticos.

O AWS CloudFormation faz alterações na pilha somente quando você decide executar o conjunto de alterações.

Os conjuntos de alterações permitem visualizar como as alterações propostas em uma pilha

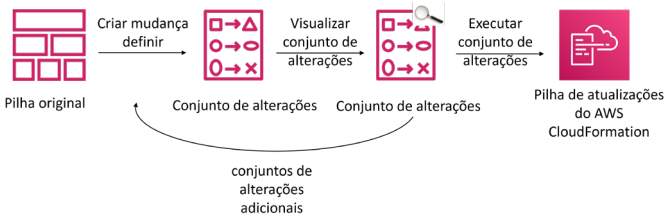
podem impactar seus recursos em execução. Por exemplo, você pode determinar se suas alterações excluirão ou substituirão recursos críticos. O AWS CloudFormation faz alterações em sua pilha somente quando você decide executar o conjunto de alterações, permitindo que você decida se deseja prosseguir com as alterações propostas ou explorar outras alterações criando outro conjunto de alterações.

Os conjuntos de alterações não indicam se o AWS CloudFormation atualizará uma pilha com êxito. Por exemplo, se estiver atualizando um recurso que não suporta atualizações, um conjunto de alterações não verificará se ultrapassará um limite de conta. Além disso, ter permissões insuficientes para modificar um recurso pode causar falha na atualização da pilha.

Se uma atualização falhar, o AWS CloudFormation tentará reverter os recursos ao estado original.

Usando conjuntos de alterações para atualizar uma pilha

Figura – Usando conjuntos de alterações para atualizar uma pilha



Fonte: elaborado pelo autor.

O diagrama ilustra o fluxo de trabalho básico de execução de conjuntos de alterações.

1. Crie um conjunto de alterações enviando alterações para a pilha que você deseja atualizar.

2. Visualize o conjunto de alterações para identificar quais configurações de pilha e recursos serão alterados.

3. Se quiser considerar outras alterações antes de decidir quais alterações fazer, crie conjuntos de alterações adicionais.

4. Execute o conjunto de alterações. O AWS CloudFormation atualiza sua pilha com essas alterações.

Mais informações (disponíveis em: http://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/using-cfn-updating-stacks-changesets.html).

Criando um conjunto de alterações

Execute o **AWS** comando **create-change-set do cloudformation**

```
Terminal X
>> aws cloudformation create-change-set \
       --stack-name NetworkStack \
       --change-set-name SampleChangeSet \
       --use-previous-template \
       --parameters \
       ParameterKey="InstanceType",UsePreviousValue=true
              ParameterKey="KeyPairName",UsePreviousValue=true
       ParameterKey="Purpose",ParameterValue="production"
```

O exemplo mostrado cria um conjunto de alterações chamado SampleChangeSet para a pilha SampleStack.

Observação: se o modelo incluir um parâmetro DefaultValue e você não especificar UsePreviousValue=true ou um valor específico, o AWS CloudFormation usará o valor padrão no modelo.

Detectando desvios em pilhas

Status da configuração real da pilha em comparação com a configuração esperada:

- DRIFTED
- NOT_CHECKED
- IN_SYNC

Mesmo quando você gerencia seus recursos por meio do AWS CloudFormation, os usuários podem alterar esses recursos **fora** do AWS CloudFormation. Os usuários podem editar recursos diretamente usando o serviço subjacente que criou o recurso. Mas o que significa *drift* (desvio)? A detecção de desvios permite detectar se a configuração real de uma pilha difere, ou desviou, da configuração esperada.

Use o AWS CloudFormation para detectar desvios em uma pilha inteira ou em recursos individuais dentro dela. Considera-se que um recurso sofreu desvio se qualquer um de seus valores de propriedade reais diferir dos valores de propriedade esperados. Isso inclui casos em que uma propriedade ou um recurso foi excluído.

Uma pilha é considerada desviada se um ou mais de seus recursos sofreram desvio.

Como o AWS CloudFormation detecta desvios?

Para determinar se um recurso foi desviado, o AWS CloudFormation determina os valores esperados das propriedades do recurso, conforme definido no modelo de pilha e quaisquer valores especificados como parâmetros do modelo. O AWS CloudFormation então compara esses valores esperados com os valores reais dessas propriedades de recurso conforme elas existem atualmente na pilha. Considera-se que um recurso foi desviado se uma ou mais de suas propriedades foram excluídas ou tiveram seu valor alterado. Ao detectar desvios em uma pilha, o AWS CloudFormation não detecta desvios em nenhuma pilha aninhada que pertença a essa pilha. Em vez disso, você pode iniciar uma operação de detecção de desvios diretamente na pilha aninhada.

Status dos desvios

- **DRIFTED:** A pilha difere da configuração de modelo esperada. Uma pilha é considerada

desviada se um ou mais de seus recursos foram desviados.

- **NOT_CHECKED:** o AWS CloudFormation não verificou se a pilha difere da configuração de modelo esperada.
- **IN_SYNC:** A configuração real da pilha corresponde à configuração de modelo esperada.

Scripts auxiliares do AWS CloudFormation

Use estes *scripts* para instalar *software* e iniciar serviços enquanto cria sua pilha:

- cfn-init
- cfn-signal
- cfn-get-metadata
- cfn-hup

O AWS CloudFormation fornece os seguintes *scripts* auxiliares Python. Você pode usar estes *scripts* para instalar *software* e iniciar serviços em uma instância do Amazon EC2 criada como parte de sua pilha:

- **cfn-init:** use para recuperar e interpretar metadados de recursos, instalar pacotes, criar arquivos e iniciar serviços. Normalmente implementado como parte dos dados do usuário;
- **cfn -signal:** use para sinalizar com CreationPolicy ou WaitCondition, para que você possa sincronizar outros recursos na pilha quando o recurso ou aplicativo de pré-requisito estiver pronto. Ele fornece um sinal de sucesso ou não;
- **cfn -get-metadata:** use para recuperar metadados de um recurso ou caminho para uma chave específica;
- **cfn-hup:** use para verificar atualizações nos metadados e executar ganchos personalizados quando alterações forem detectadas.

Você chama os *scripts* diretamente do seu modelo. Os *scripts* funcionam com metadados de recursos definidos no mesmo modelo. Os *scripts* são executados na instância do Amazon EC2 durante o processo de criação da pilha. Lembre-se de que os *scripts* não são executados por padrão. Você deve incluir chamadas em

seu modelo para executar *scripts* auxiliares específicos.

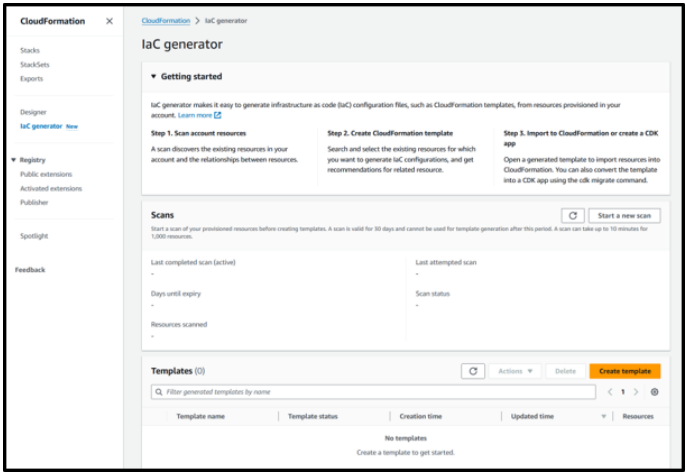
Pergunta: o que acontece quando altero os metadados no meu modelo e atualizo uma pilha existente?

Resposta: por padrão, nada acontece. Somente quando você executa cfn-init ou quando cfn-hup está configurado para monitorar alterações de metadados, alguma ação ocorrerá.

Gerador IaC

Gere modelos do CloudFormation a partir de recursos já provisionados em sua conta

- Digitalizar os recursos da conta no CloudFormation
- Criar modelo CloudFormation
- Adicionar recursos verificados ao modelo
- Importe recursos para o CloudFormation a partir do modelo



Informações adicionais (disponíveis em: https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/generate-IaC.html e https://aws.amazon.com/blogs/devops/import-entire-applications -into-aws-cloudformation/).



Conceitos de automação

Provisionamento – Infraestrutura como código

SAM

AWS SAM (modelo de aplicativo sem servidor AWS)

AWS SAM

Estrutura de código aberto usada para construir aplicativos sem servidor

Consiste nos seguintes componentes:

- especificação do modelo AWS SAM;
- interface de linha de comando do AWS SAM (AWS SAM CLI).



O *AWS Serverless Application Model* (AWS SAM) é uma estrutura de código aberto que você pode usar para criar aplicativos sem servidor na AWS.

Lembre-se: um **aplicativo sem servidor** é uma combinação de funções Lambda, fontes de eventos e outros recursos que trabalham juntos para executar tarefas. Uma aplicação sem servidor é mais do que apenas uma função Lambda: ela pode incluir recursos adicionais, como operações de API, bancos de dados e mapeamentos de fontes de eventos.

Você pode usar o AWS SAM para definir seus aplicativos sem servidor. AWS SAM consiste nos seguintes componentes:

- **Especificação do modelo AWS SAM:** você usa esta especificação para definir seu aplicativo sem servidor. Ele fornece uma

sintaxe simples e limpa para descrever as funções, APIs, permissões, configurações e eventos que compõem um aplicativo sem servidor. Você usa um arquivo de modelo do AWS SAM para operar em uma entidade única, implantável e com versão que é seu aplicativo sem servidor. Para a especificação completa do modelo AWS SAM, consulte "Especificação do modelo de aplicativo sem servidor AWS" (disponível em: https://github.com/awslabs/serverless-application-model/blob/master/versions/2016-10-31.md).

- **Interface de linha de comando do AWS SAM (AWS SAM CLI):** você usa essa ferramenta para criar aplicativos sem servidor definidos por modelos do AWS SAM. O A CLI fornece comandos que permitem verificar se os arquivos de modelo do AWS SAM foram gravados de acordo com a especificação, invocar funções do Lambda localmente, executar e depurar funções do Lambda, empacotar e implantar aplicativos sem servidor na Nuvem AWS, e assim por diante. Para mais detalhes sobre como usar a CLI do AWS SAM, incluindo a

referência completa de comandos da CLI do AWS SAM, consulte "CLI do AWS SAM" (disponível em: https://docs.aws.amazon.com/serverless-application-model/latest/developerguide/serverless -sam-reference.html#serverless-sam-cli).

Benefícios de usar AWS SAM

- Configuração de implantação única
- Práticas recomendadas integradas
- Depuração e testes locais
- Integração profunda com ferramentas de desenvolvimento
  - AWS Cloud9 IDE, AWS CodeBuild, AWS CodeDeploy, AWS CodePipeline
- Compila para o modelo AWS CloudFormation

Como o AWS SAM se integra a outros serviços da AWS, a criação de aplicativos sem servidor com o AWS SAM oferece os seguintes benefícios:

- **Configuração de implantação única.** Usando o AWS SAM, você pode organizar

componentes e recursos relacionados e operar em uma única pilha. Você pode usar o AWS SAM para compartilhar configurações (como memória e tempos limite) entre recursos. Você também pode usá-lo para implantar todos os recursos relacionados juntos como uma entidade única com versão.

- **Melhores práticas integradas.** Você pode usar o AWS SAM para definir e implantar sua infraestrutura como config. Isso possibilita que você use e aplique práticas recomendadas, como revisões de código. Além disso, com algumas linhas de configuração, você pode habilitar implantações seguras por meio do AWS CodeDeploy e habilitar o rastreamento usando o AWS X-Ray.
- **Depuração e testes locais.** A CLI do AWS SAM permite criar, testar e depurar localmente aplicativos sem servidor definidos por modelos do AWS SAM. A CLI fornece localmente um ambiente de tempo de execução semelhante ao Lambda. Ele ajuda você a detectar problemas antecipadamente, fornecendo paridade

com o ambiente de tempo de execução real do Lambda. Para percorrer e depurar seu código para entender o que ele está fazendo, você pode usar o AWS SAM com kits de ferramentas da AWS, como

- Kit de ferramentas da AWS para JetBrains
- Kit de ferramentas AWS para PyCharm
- Kit de ferramentas da AWS para IntelliJ
- AWS Toolkit para código do Visual Studio

Isso estreita o ciclo de feedback, possibilitando que você encontre e solucione problemas que possa encontrar na nuvem.

- **Integração profunda com ferramentas de desenvolvimento.** Você pode usar o AWS SAM com um conjunto de ferramentas da AWS para criar aplicativos sem servidor. Você pode descobrir novos aplicativos no AWS Serverless Application Repository. Para criar, testar e depurar aplicativos sem servidor baseados em AWS SAM, você pode usar o IDE do AWS Cloud9.

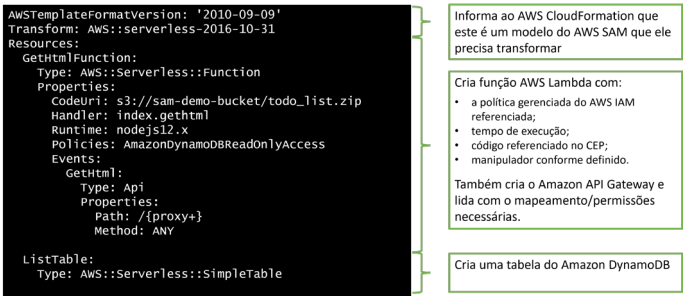
Para criar um pipeline de implantação para seus aplicativos sem servidor, você pode usar

CodeBuild, CodeDeploy e CodePipeline. Você também pode usar o AWS CodeStar para começar a usar uma estrutura de projeto, um repositório de código e um pipeline de CI/CD configurado automaticamente para você. Para implantar seu aplicativo sem servidor, você pode usar o plugin Jenkins. Para criar aplicativos prontos para produção, use o *kit* de ferramentas Stackery.io.

- **Compila para o modelo AWS CloudFormation:** como o AWS SAM é uma extensão do AWS CloudFormation, você obtém os recursos de implantação confiáveis do AWS CloudFormation. Você pode definir recursos usando o AWS CloudFormation no modelo do AWS SAM. Além disso, você pode usar o conjunto completo de recursos, funções intrínsecas e outros recursos de modelo disponíveis no AWS CloudFormation.

Modelo AWS SAM

Figura – Modelo AWS SAM



Fonte: elaborado pelo autor

Se você criar um aplicativo sem servidor usando o AWS SAM, seu objetivo principal é construir um arquivo de modelo do AWS SAM que represente a arquitetura do seu aplicativo sem servidor. O arquivo de modelo do AWS SAM é um arquivo de configuração YAML ou JSON que segue a especificação do *AWS Serverless Application Model* de código aberto. Você usa o modelo para declarar todos os recursos da AWS que compõem seu aplicativo sem servidor.

Os modelos AWS SAM são uma extensão dos modelos AWS CloudFormation. Ou seja, qualquer recurso que você possa declarar em um modelo AWS CloudFormation, também poderá declarar em um modelo AWS SAM. A vantagem de usar o

AWS SAM em comparação com um modelo cfn é que, com um número menor de linhas de sintaxe do AWS SAM, você pode criar vários recursos. O AWS SAM permite agilizar o processo.

Declaração de recursos sem servidor do AWS SAM

- AWS::Sem servidor::API
- AWS::Serverless::Aplicativo
- AWS::Serverless::Function
- AWS::Serverless::LayerVersion
- AWS::Sem servidor::SimpleTable
- AWS::Serverless::StateMachine

API

Esse tipo de recurso descreve um recurso do API *Gateway*. É útil para casos de uso avançados em que você deseja total controle e flexibilidade ao configurar suas APIs. Para a maioria dos cenários, recomendamos que você crie APIs especificando esse tipo de recurso como uma fonte de evento do seu recurso AWS::Serverless::Function.

Aplicativo

Esse tipo de recurso incorpora um aplicativo sem servidor do *AWS Serverless Application Repository* ou de um *bucket* do Amazon S3 como um aplicativo aninhado. Os aplicativos aninhados são implantados como pilhas aninhadas, que podem conter vários outros recursos.

Função

Esse tipo de recurso descreve informações de configuração para criar uma função Lambda. Você pode descrever qualquer fonte de eventos que deseja anexar à função Lambda, como Amazon S3, *Amazon DynamoDB Streams* e *Amazon Kinesis Data Streams*.

Versão da camada

Esse tipo de recurso cria uma versão da camada Lambda (LayerVersion) que contém bibliotecas ou código de tempo de execução necessário para uma função Lambda. Quando uma versão da camada sem servidor é transformada, o AWS SAM também transforma o ID lógico do recurso para que as versões antigas da camada não sejam excluídas automaticamente pelo AWS CloudFormation quando o recurso é atualizado.

Tabela simples

Esse tipo de recurso fornece sintaxe simples para descrever como criar tabelas do DynamoDB.

Máquina de estado

Esse tipo de recurso cria uma máquina de estado do *AWS Step Functions*, que você pode usar para orquestrar funções do AWS Lambda e outros recursos da AWS para formar fluxos de trabalho complexos e robustos.

Usar a CLI do AWS SAM para empacotar e implantar modelos

- pip install-user
- Sam init
- Sam build
- package
- Sam deploy

CodePipeline Use ações de implantação do AWS CloudFormation com qualquer AWS SAM aplicativo. ***Jenkins*** UsarAWS SAM Plug-in CLI.

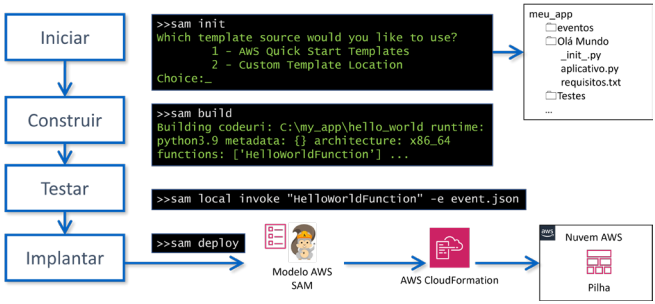
Aprofundaremos cada um desses comandos a seguir.

Para o CodePipeline, *sam package* e *sam deploy* são *wrappers* em torno dos conjuntos de alterações do AWS CloudFormation. Você ainda usaria as ações do AWS CloudFormation com qualquer modelo AWS SAM formulários. Para o *Jenkins*, você instalaria o AWS SAM Plug-in CLI em seu servidor *Jenkins* e, em seguida, implantaria no Lambda com seus aplicativos sem servidor.

Mais informações sobre comandos CLI do AWS SAM (disponíveis em: https://docs.aws.amazon.com/serverless-application-model/latest/developerguide/serverless-sam-cli-command-reference.html).

Fluxo de trabalho do AWS SAM

Figura – Fluxo de trabalho do AWS SAM



Fonte: elaborado pelo autor.

- **Init:** para inicializar um novo projeto AWS SAM, use o comando sam init. Você pode escolher um modelo de aplicativo integrado ou um modelo personalizado.
- **Build:** para construir seu aplicativo AWS SAM, execute o comando sam *build*. Você pode construir dentro de um contêiner.
- **Teste:** para executar seu aplicativo AWS SAM localmente para fins de teste, execute o comando sam local invocar.
- **Implementar:** para implementar por meio de um *prompt* interativo, se você estiver usando um arquivo de configuração, execute o comando *sam deploy*.

Inicializar aplicativos

Para inicializar um aplicativo sem servidor com um modelo AWS SAM, use sam iniciar.

```
Initializes a new SAM project using Python 3.6 default template runtime
$ sam init --runtime python3.6
Initializes a new SAM project using custom template in a Zipfile
$ sam init --location /path/to/template.zip
$ sam init --location https://example.com/path/to/template.zip
Initializes a new SAM project using custom template in a local path
$ sam init --location /path/to/template/folder
```

Inicialize um aplicativo sem servidor com um modelo AWS SAM. O modelo fornece uma estrutura de pastas para suas funções do Lambda e está conectado a uma fonte de eventos, como APIs, buckets S3 ou tabelas do DynamoDB. Esse aplicativo inclui tudo que você precisa para começar e, eventualmente, estendê-lo para um aplicativo em escala de produção.

Construindo aplicações com AWS SAM

Para compilar dependências para funções Lambda em Python, use **sam build.**

Para uma invocação local única de uma função do AWS Lambda, use a **invocação local sam** subcomando.

```
# Run the build process inside an AWS Lambda-like Docker container
$ sam build --use-container

# Build and run your functions locally
$ sam build && sam local invoke
```

Você pode usar o comando *sam build* para compilar dependências para funções Lambda escritas em Python. Por exemplo, se você escrever código que usa pacotes Python, como uma biblioteca gráfica para processamento de imagens, crie um pacote de implantação que

funcione no Amazon Linux Amazon Machine Image (AMI). Você pode usar o *sam build* para criar artefatos de implantação direcionados ao ambiente de tempo de execução do Lambda, para que as funções que você cria localmente sejam executadas em um ambiente semelhante na Nuvem AWS.

Se a sua função Lambda depender de pacotes que possuem programas compilados nativamente, você poderá usar o sinalizador --use-container. O --use-container flag compila suas funções localmente em um ambiente semelhante ao Lambda, para que estejam no formato correto quando você as implanta na Nuvem AWS.

Implantando um aplicativo

O uso **do pacote sam** cria automaticamente artefatos de implantação que você pode implantar no Lambda.

Para implantar um aplicativo AWS SAM, use **sam deploy**.

- Implanta seu aplicativo no AWS CloudFormation como uma pilha

```
# Build and package for deployment
$ sam build && sam package --s3-bucket <bucketname>

$ sam-app> sam deploy \
    --template-file packaged.yaml \
    --stack-name sam-app \
    --capabilities CAPABILITY_IAM \
    --region us-east-1
```

O comando *sam build* itera pelas funções do seu aplicativo, procura um arquivo de manifesto (como requisitos.txt) que contém as dependências e cria automaticamente artefatos de implantação que você pode implantar no Lambda usando os comandos *sam package* e *sam deploy*.

Neste exemplo, usando a CLI do AWS SAM, execute a implantação Comando CLI para implementar todos os recursos definidos no modelo. No comando, o parâmetro --capabilities permite que o AWS CloudFormation crie uma função do IAM. O AWS CloudFormation cria os recursos da AWS definidos no modelo. Você pode acessar os nomes desses recursos no console do AWS CloudFormation.

Invocar funções localmente

Para invocar sua função localmente, use **sam local invocar**.

- Fornece ID lógico de função e arquivo de evento

```
# Invoking function with event file
$ sam local invoke "Ratings" -e event.json

# Invoking function with event via stdin
$ echo '{"message": "Hey, are you there?" }' | sam local invoke "Ratings"

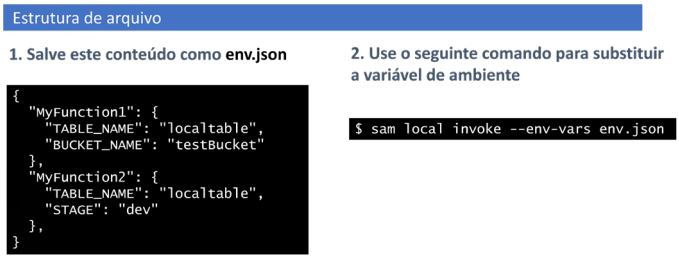
# For more options
$ sam local invoke --help
```

Você pode invocar sua função localmente usando o comando “sam local invoke”, fornecendo o ID lógico da função e um arquivo de evento. Alternativamente, o comando “sam local invoke” também aceita o evento via stdin.

O comando “sam local invoke” descrito nesta seção corresponde ao comando “aws lambda invoke” da AWS CLI. Você pode usar este último comando para invocar uma função Lambda que já foi implantada na Nuvem AWS.

Você deve executar o comando “sam local invoke” no diretório do projeto que contém a função que deseja invocar.

Invocando funções localmente: arquivo de variável de ambiente



Arquivo de variável de ambiente

Você pode usar o argumento --env-vars com os comandos invocar ou start-api. Faça isso para fornecer um arquivo JSON que contém valores para substituir as variáveis de ambiente já definidas em seu modelo de função. Estruture o arquivo conforme mostrado.

Então, se você salvar o conteúdo em um arquivo chamado env.json, o comando a seguir usará esse arquivo para substituir as variáveis de ambiente incluídas.

Repositório de aplicativos sem servidor da AWS

Figura – Repositório de aplicativos sem servidor da AWS



Fonte: elaborado pelo autor.

O *AWS Serverless Application Repository* é um repositório gerenciado para aplicativos sem servidor. Ele permite que equipes, organizações e desenvolvedores individuais armazenem e compartilhem aplicativos reutilizáveis e montem e implantem facilmente arquiteturas sem servidor de novas maneiras poderosas.

Usando o *Serverless Application Repository*, você não precisa clonar, criar, empacotar ou publicar o código-fonte na AWS antes de implantá-lo. Em vez disso, você pode usar aplicativos pré-construídos do *Serverless Application Repository* em suas arquiteturas sem servidor, ajudando você e suas equipes a reduzir o trabalho duplicado, garantir as melhores práticas organizacionais e chegar ao mercado mais rapidamente. A

integração com o *AWS Identity and Access Management* (IAM) fornece controle em nível de recurso de cada aplicativo, permitindo que você compartilhe aplicativos publicamente com todos ou de forma privada com contas específicas da AWS.

Cada aplicativo é empacotado com um modelo *AWS Serverless Application Model* (AWS SAM) que define os recursos da AWS usados. Os aplicativos compartilhados publicamente também incluem um *link* para o código-fonte do aplicativo. Não há cobrança adicional para usar o *Serverless Application Repository* – você paga apenas pelos recursos da AWS usados nos aplicativos implantados.

Para mais informações, consulte "O que é o AWS Serverless Application Repository?" (disponível em: https://docs.aws.amazon.com/serverlessrepo/latest/devguide/what-is-serverlessrepo.html).

Terraform

Prática – Terraform

Terraform

- Básico (HCL, *providers*)
- Planejamento (*Plan*)
- Aplicação (*Apply*)
- Contagem (*Count*)
- Módulos (*Modules*)
- Fontes de dados (*Data Sources*)
- Estado (*State*)
- Áreas de trabalho (*Workspaces*)

Prática – Terraform – Básico

A sintaxe das configurações do Terraform é chamada de **HachiCorp Configuration Language (HCL)**. Destina-se a encontrar um equilíbrio entre legível e editável, além de ser *machine friendly*. Para facilidade de uso, o Terraform também pode ler configurações JSON. Para configurações gerais do Terraform, no entanto, recomendamos o uso da sintaxe Terraform da HCL.

Arquivo.tf

```
# An AMI
variable "ami" {
 description = "the AMI to use"
}

/* A multi
line comment. */
resource "aws_instance" "web" {
 ami = "${var.ami}"
 count = 2
 source_dest_check = false

 connection {
  user = "root"
 }
}
```

Variáveis

```
variable "key" {
 type = "string"
}

variable "images" {
 type = "map"

 default = {
  us-east-1 = "image-1234"
  us-west-2 = "image-4567"
 }
}

variable "zones" {
 default = ["us-east-1a", "us-east-1b"]
}
```

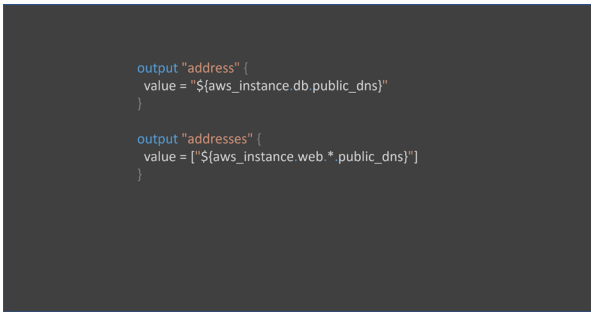
Arquivo de variáveis (terraform.tfvars)

```
foo = "bar"
xyz = "abc"

somelist = [
 "one",
 "two",
]

somemap = {
 foo = "bar"
 bax = "qux"
}
```

Output values



Planejamento (*Plan*)

O comando *terraform plan* é usado para criar um plano de execução. O Terraform executa uma atualização, a menos que ela seja explicitamente desabilitada, e depois determina quais ações são necessárias para atingir o estado desejado especificado nos arquivos de configuração.

Esse comando é uma maneira conveniente de verificar se o plano de execução de um conjunto de alterações corresponde às suas expectativas sem fazer alterações nos recursos reais ou no estado. Por exemplo, o plano de execução gerado pelo comando *terraform plan* pode ser executado antes de confirmar uma alteração no controle de versão, para aumentar a confiança de que ele se comportará conforme o esperado.

```
$ terraform plan -out=terraform.tfplan terraform/
Refreshing Terraform state prior to plan...


The Terraform execution plan has been generated and is shown below.
Resources are shown in alphabetical order for quick scanning. Green resources
will be created (or destroyed and then created if an existing resource
exists), yellow resources are being changed in-place, and red resources
will be destroyed.

Your plan was also saved to the path below. Call the "apply" subcommand
with this plan file and Terraform will exactly execute this execution
plan.

Path: terraform.tfplan

+ digitalocean_droplet.app
    backups:              "" => "1"
    image:                "" => "ubuntu-14-04-x64"
    ipv4_address:         "" => "<computed>"
    ipv4_address_private: "" => "<computed>"
    ipv6:                 "" => "1"
    ipv6_address:         "" => "<computed>"
    ipv6_address_private: "" => "<computed>"
    locked:               "" => "<computed>"
    name:                 "" => "app-01"
    region:               "" => "lon1"
    size:                 "" => "512mb"
    status:               "" => "<computed>"

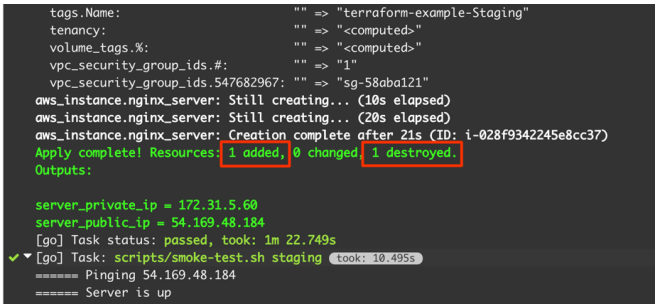
+ digitalocean_droplet.hosting
    backups:              "" => "1"
    image:                "" => "ubuntu-14-04-x64"
    ipv4_address:         "" => "<computed>"
    ipv4_address_private: "" => "<computed>"
    ipv6:                 "" => "1"
    ipv6_address:         "" => "<computed>"
    ipv6_address_private: "" => "<computed>"
    locked:               "" => "<computed>"
    name:                 "" => "hosting-01"
    region:               "" => "lon1"
    size:                 "" => "512mb"
    status:               "" => "<computed>"


Plan: 2 to add, 0 to change, 0 to destroy.
```

O argumento opcional –out pode ser usado para salvar o plano gerado em um arquivo para execução posterior com *terraform apply*, o que pode ser útil ao executar o Terraform na automação.

Aplicação (*Apply*)

O comando *terraform apply* é utilizado para executar as ações previstas no plano de execução gerado anteriormente pelo comando *terraform plan*. Ao ser executado, o Terraform cria, altera ou remove recursos de infraestrutura conforme definido nos arquivos de configuração e validado durante a etapa de planejamento.



Exemplo da saída de comando *terraform apply*, demonstrando a execução das alterações previamente aprovadas no plano de execução e a atualização da infraestrutura para o estado desejado.

Por padrão, aplique a varredura do diretório atual para a configuração e aplique as alterações adequadamente. No entanto, um caminho para outra configuração ou um plano de execução pode ser fornecido. Arquivos de planos de

execução explícitos podem ser usados para dividir o plano e aplicá-lo em etapas separadas nos sistemas de automação.

Contagem (*Count*)

O parâmetro *count* nos recursos pode simplificar as configurações e permitir dimensionar recursos simplesmente incrementando um número.

Além disso, variáveis podem ser usadas para expandir uma lista de recursos para uso em outro lugar.

```
# Specify the provider and access details
provider "aws" {
  region = "${var.aws_region}"
}

resource "aws_instance" "web" {
  instance_type = "m1.small"
  ami = "${lookup(var.aws_amis, var.aws_region)}"

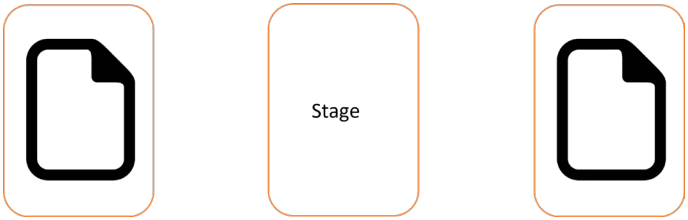
  # This will create 4 instances
  count = 4
}
```

Módulos (*Modules*)

É tentador definir todo o seu código de infraestrutura em um arquivo / pasta...

Desvantagens: roda mais devagar; mais difícil de entender; mais difícil de rever (planejar saída ilegível); mais difícil de testar; mais difícil de reutilizar código; precisa de permissões de administrador; equipe concorrente limitada a 1...

Além disso, um erro em qualquer lugar poderia quebrar tudo!

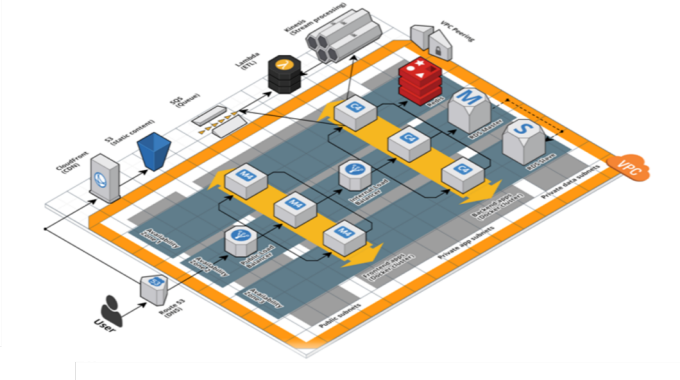


O que você realmente quer é **isolamento** para cada ambiente.



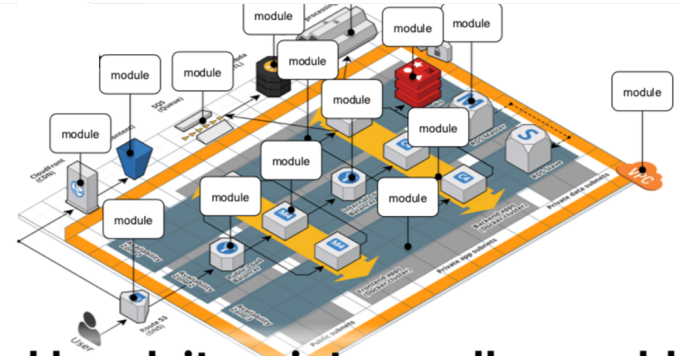
O mesmo princípio de isolamento pode ser aplicado aos componentes individuais da arquitetura (VPC, *Front-end*, Banco de Dados etc.).

Figura - Prática – Terraform – Módulos (Modules)



Fonte: elaborado pelo autor.

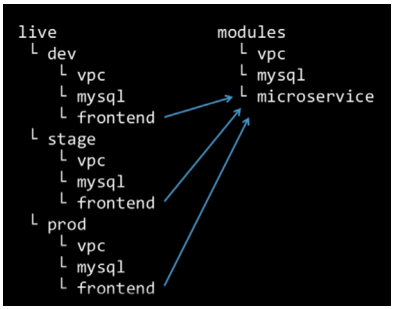
Figura - Prática – Terraform – Módulos (Modules)



Fonte: elaborado pelo autor.

```
live
 └ dev
 └ stage
 └ prod
```






```
provider "aws" {
  region = "us-east-1"
}

module "consul" {
  source = "hashicorp/consul/aws"

  num_servers = "3"
}
```

```
module "database" {
  source = "github.com/terraform-community-modules/tf_aws_rds"

  # DB Instance Inputs
  rds_instance_identifier = "${terraform.workspace}-${var.app}-db"
  rds_allocated_storage   = "${var.database_size}"

  database_name     = "${var.database_name}"
  database_user     = "${var.database_user}"
  database_password = "${var.database_password}"

  # DB Subnet Inputs
  subnets = ["${aws_subnet.dbnet.*.id}"]
  rds_vpc_id = "${data.aws_vpc.app.id}"

  tags {
    Name = "${terraform.workspace} - ${var.app} - DB"
  }
}
```

Data sources (Fontes de dados)

As *data sources* permitem que os dados sejam consultados ou calculados para uso em outras partes da configuração do Terraform. O uso de *data sources* permite que uma configuração Terraform construa informações definidas fora do Terraform, ou definidas por outra configuração separada do Terraform.

Os provedores são responsáveis, no Terraform, pela definição e implementação de *data sources*. Enquanto um recurso faz com que o Terraform crie e gerencie um novo componente de infraestrutura, as *data sources* representam fontes de dados somente leitura de dados preexistentes, ou calculam novos valores rapidamente dentro do próprio Terraform.

```
# Find the latest available AMI that is tagged with Component = web
data "aws_ami" "web" {
  filter {
    name = "state"
    values = ["available"]
  }

  filter {
    name = "tag:Component"
    values = ["web"]
  }

  most_recent = true
}
```

Estado *(State)*

O estado é um requisito necessário para que o Terraform funcione. Muitas vezes, pergunta-se se é possível que o Terraform funcione sem estado, ou que o Terraform não use o estado e apenas inspecione os recursos da nuvem a cada execução.

O Terraform requer algum tipo de banco de dados para mapear a configuração do Terraform para o mundo real. Quando você tem um recurso de “aws_instance” “foo” em sua configuração, o Terraform usa esse mapeamento para saber que a instância i-abcd1234 é representada por esse recurso.

Para alguns provedores como o AWS, o Terraform teoricamente poderia usar algo como *tags* da AWS. Os primeiros protótipos do Terraform na verdade não tinham arquivos de estado e empregavam esse método. No entanto, rapidamente surgiram problemas. A primeira grande questão foi simples: nem todos os recursos suportam *tags* e nem todos os provedores de nuvem oferecem suporte a *tags*.

Portanto, para mapear a configuração para recursos no mundo real, o Terraform usa sua própria estrutura de estado.

Na configuração padrão, o Terraform armazena o estado em um arquivo no diretório de trabalho atual onde o Terraform foi executado. Não há problema em começar, mas ao usar o Terraform em uma equipe, é importante que todos trabalhem com o mesmo estado, para que as operações sejam aplicadas aos mesmos objetos remotos.

Estado remoto é a solução recomendada para esse problema. Com um *backend* de estado completo, o Terraform pode usar o bloqueio remoto como uma medida para evitar que dois ou mais usuários diferentes executem acidentalmente o Terraform ao mesmo tempo, e assim garantir que cada execução do Terraform comece com o estado atualizado mais recente.

Por padrão, o Terraform armazena o estado localmente em um arquivo chamado terraform.tfstate. Ao trabalhar com o Terraform em uma equipe, o uso de um arquivo local torna o uso do Terraform complicado, pois cada usuário deve ter

sempre os dados de estado mais recentes antes de executar o Terraform e garantir que ninguém execute o Terraform ao mesmo tempo.

Com estado remoto, o Terraform grava os dados do estado em um armazenamento de dados remoto, que pode ser compartilhado entre todos os membros de uma equipe. O Terraform suporta o estado de armazenamento no Terraform Enterprise, HashiCorp Consul, Amazon S3 e muito mais.

Estado remoto é uma característica dos *backends*. Configurar e usar *backends* remotos é fácil e você pode começar rapidamente com o estado remoto. Se você quiser migrar de volta para o estado local, os *backends* também facilitam isso.

Workspaces (Áreas de trabalho)

Cada configuração do Terraform possui um *backend* associado que define como as operações são executadas e onde os dados persistentes, como o estado Terraform, são armazenados.

Os dados persistentes armazenados no *backend* pertencem a um *workspace*. Inicialmente, o *backend* tem apenas uma área de trabalho,

chamada "*default*", e, portanto, há apenas um estado Terraform associado a essa configuração.

Certos *backends* suportam vários *workspaces* nomeados, permitindo que vários estados sejam associados a uma única configuração. A configuração ainda tem apenas um *backend*, mas várias instâncias distintas dessa configuração devem ser implantadas sem configurar um novo *backend* ou alterar as credenciais de autenticação.

Terraform começa com um único *workspace* chamado "padrão". Essa área de trabalho é especial porque é o padrão e também porque nunca pode ser excluída. Se você nunca usou workplaces explicitamente, só trabalhou na área de trabalho "padrão".

Os *workspaces* são gerenciados com o conjunto de comandos do terraform workspace. Para criar um novo workspace e alternar para ele, você pode usar o terraform workspace new; para alternar workspace, você pode usar o terraform workspace select; etc.